

Data Structures

CMSC 27200: Theory of Algorithms

January 17, 2025

Corresponding section(s) of Kleinberg-Tardos: 2.2

Data structures are structures for efficiently storing the data we need for our algorithms. Data structures like arrays, linked lists, heaps, binary search trees, and hash tables specify exactly how the data is stored. Abstract data types specify which operations may be performed on the data without specifying exactly how these operations are implemented.

1 Arrays and Linked Lists

Two fundamental data structures are arrays and linked lists.

An array of length n has n elements $A[0], A[1], \dots, A[n - 1]$ (most programming languages start with 0 as the first index of the array) which can be accessed and modified in $O(1)$ time.

1. $A[i]$: Gives the element at index i of A .
2. $A[i] = x$: Sets the element at index i of A to be x .

Arrays have the following advantages and disadvantages.

1. Advantages: It is easy to access and change the data in an array.
2. Disadvantages: You have to keep track of where everything in the array is stored yourself. Also, you have to decide on the size of the array beforehand.

For many algorithms, it is useful to take the elements in an array and sort them. This can be done in $O(n \log n)$ time using mergesort or other advanced sorting algorithms.

1. Advantages: Using a sorted array, it is easy to go through the elements in order. Moreover, we can search for an element in a sorted array in time $O(\log n)$ using binary search.
2. Disadvantage: A sorted array is a static data structure. In other words, we can't modify the data in a sorted array without rebuilding it from scratch.

A linked list is a set of nodes where each node points to the next node. More precisely, a linked list keeps track of the following.

1. We store the head node (and possibly the tail node) of the linked list.
2. Each node contains an object (i.e., the data we're trying to store) and a pointer to the next node. In a doubly-linked list, each node also contains a pointer to the previous node.

This allows us to start from the head node and go through the entire linked list.

1. Advantages: It is easy to add elements to either the head or the tail of a linked list. For a doubly linked list, it is easy to remove an element from anywhere in the list if we have a pointer to the node containing the element we want to remove. It is not necessary to decide on the length of the linked list ahead of time.
2. Disadvantages: The middle of a linked list is not easily accessible. If we want to search the list to see if it contains a given element, this requires going through the entire list.

2 Stacks and Queues

A stack is an abstract data type with the following operations:

1. Push(x): Add x to the top of the stack.
2. Pop: Remove the top element of the stack and return it. If the stack is empty, return Null instead.
3. Peek: Return the top element of the stack (or Null if the stack is empty).

A stack can be implemented using either an array or a linked list.

To implement a stack with an array, we have the stack keep track of its current height $j \geq 0$ and an array A which stores the elements $A[0], \dots, A[j-1]$ of the stack.

1. Peek: If $j \geq 1$, return $A[j-1]$. If $j = 0$, return Null.
2. Pop: If $j \geq 1$, return $A[j-1]$ and decrement j . If $j = 0$, return Null.
3. Push(x): If $j < n$, set $A[j] = x$ and increment j . If $j = n$ (i.e., the array is filled to capacity), copy the elements of A into a new array A' which is twice as large. We then set $A'[j] = x$, increment j , and replace A with A' .

One way to implement a stack with a linked list is as follows. We use a linked list where we keep track of the head but not the tail.

1. Peek: Return the element at the head of the linked list. If the linked list is empty, return Null.
2. Pop: If the linked list is non-empty, return the element at the current head and set the new head to be the next node after the head (or Null if there are no other nodes in the linked list). If the linked list is empty, return Null.
3. Push(x): Create a new node with element x which points to the current head (or Null if the linked list is empty) and set the new head to be this node.

A queue is an abstract data type with the following operations:

1. Enqueue(x): Add x to the back of the queue.
2. Dequeue: Remove the element at the front of the queue and return it. If the queue is empty, return Null.
3. Peek: Return the element at the front of the queue. If the queue is empty, return null.

A queue can be implemented using either an array or a linked list.

To implement a queue with an array, we have the queue keep track of indices l and r and an array A which stores the elements of the queue. The elements of the queue are $A[l], A[l + 1], \dots, A[r - 1]$ if $l < r$ and $A[l], A[l + 1], A[n], A[0], \dots, A[r - 1]$ if $l > r$.

1. Peek: If $l \neq r$, return $A[l]$. If $l = r$, return Null.
2. Dequeue: If $l = r$, return Null. If $l \neq r$, return $A[l]$ and increment l . At this point, if $l = n$, set $l = 0$.
3. Enqueue(x): Set $A[r] = x$ and increment r . If $r = n$ then we set $r = 0$. At this point, if $l = r$ then we create a new array A' which is twice the size of A . We then copy the elements of the queue to $A'[0], \dots, A'[n - 1]$, set $l = 0$ and $r = n$, and replace A with A' .

One way to implement a queue with a linked list is as follows. We use a linked list where we keep track of both the head and the tail.

1. Peek: Return the element at the head of the linked list. If the linked list is empty, return Null.
2. Dequeue: If the linked list has two or more elements (which holds if the head and tail are distinct), return the element at the current head of the linked list and set the new head to be the the next node after the current head. If the linked list has exactly one node (which happens if the head and the tail are the same), return the element in this node and set the head and tail to be Null. If the linked list is empty, return Null.
3. Enqueue(x): If the queue is non-empty, create a new node with element x which has a Null pointer, make the current tail point to this node, and set the new tail to be this node. If the queue is empty, create a node with element x which has a Null pointer and set both the head and the tail to be this node.

For both stacks and queues, all of these operations take $O(1)$ time.

Remark 2.1. *If an array is used for the implementation then the Pop operation for the stack and the Enqueue operation for the queue take $O(1)$ amortized time as copying the elements to a new array takes $O(n)$ time but these operations are rare.*

3 Priority Queues and Heaps

A priority queue is an abstract data type which is similar to a queue except that each element has a priority and instead of taking the element at the front of the queue, we take the element with the smallest value for its priority.

1. Insert: Adds a new element to the priority queue.
2. Delete min: Removes the element with the smallest value for its priority.
3. Peek: Return the element with the smallest value for its priority without removing it.

Priority queues can be implemented using binary heaps so that every operation takes $O(\log n)$ time.

3.1 Binary min-heaps

A binary min-heap is a data structure where each node has 0, 1, or 2 children and we have the property that the value of each node is less than or equal to the values of its children. This implies that the smallest element of the heap must be at the top of the heap.

It is convenient to add the further restriction that the heap is an almost complete binary tree. In other words, if we order the nodes of a complete binary tree in order of their depth and go from left to right at each depth, we always have that the first k nodes are occupied where k is the number of elements in the heap.

Using a binary min-heap, we can implement the operations of a priority queue as follows.

1. Insert(x): Add a node with x to the next empty spot. While x is smaller than its parent, swap it with its parent.
2. Peek: Return the top element of the heap. If the heap is empty, return Null.
3. Delete: Return the top element of the heap. After doing this, move the element in the last occupied spot to the top. While this element is larger than one or both of its children, swap it with its smaller child.

If the heap is empty, return Null.

There is a nice way to implement a minimum binary heap using an array. The idea is that each $A[i]$ is a node and the left and right children of this node are $A[2i]$ and $A[2i + 1]$ respectively. The top of the heap is $A[1]$.

Using this idea, we can implement a priority queue using an array as follows. We store the current number k of elements in the heap and we store the elements of the heap in an array A of length n .

1. Peek: If $k \geq 1$, return $A[1]$. If $k = 0$, return Null.
2. Insert: Set $A[k + 1] = x$, take $j = k + 1$, and increment k . After doing this, while $j > 1$ and $A[j] < A[\lfloor \frac{j}{2} \rfloor]$, swap $A[j]$ and $A[\lfloor \frac{j}{2} \rfloor]$ and set $j = \lfloor \frac{j}{2} \rfloor$.

If $k = n - 1$, we first copy the elements of A to a new array which is twice as large.

3. Delete: Return $A[1]$, set $A[1] = A[k]$, decrement k , and set $j = 1$. After doing this, while $A[j] > A[2j]$ or $A[j] > A[2j + 1]$,

- (a) If $A[2j] \leq A[2j + 1]$ then swap $A[j]$ and $A[2j]$ and set $j = 2j$.
- (b) If $A[2j + 1] < A[2j]$ then swap $A[j]$ and $A[2j + 1]$ and set $j = 2j + 1$.

If $k = 0$ then we instead just return Null.

4 Binary trees

A binary tree is a data structure where each node has 0, 1, or 2 children and we have the property that each node is larger than its left child (if any) and smaller than its right child.

Binary trees have the advantage that it is generally easy to search for an element in a binary tree or add a new element to a binary tree.

1. To search for an element x in a binary tree, we start at the root and iteratively compare x to the current node. If the current node is x then we are done. If x is less than the current node, we go to the left child of the current node. If the current node has no left child then x is not in the tree. If x is larger than the current node, we go to the right child of the current node. If the current node has no right child then x is not in the tree.
2. To insert a new element in a binary tree, we go down the tree to find the right spot and add a new node there.

If the binary tree is reasonably balanced then searching for elements and inserting elements takes $O(\log n)$ time. However, in the worst case, these operations can take $O(n)$ time.

There are sophisticated types of binary trees such as red-black trees which automatically balance themselves and also allow elements to be deleted. For these trees, all of these operations take $O(\log n)$ time.

5 Table of Running Times for Data Structures

Data structure	Search	Find min	Insertion	Deletion
Sorted array	$O(\log n)$	$O(1)$	Not supported	Not supported
Linked list	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Binary min-heap	Not supported	$O(1)$	$O(\log n)$	$O(\log n)$
Red-black trees	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

Remark 5.1. In practice, hash tables are very efficient for search, insertion, and deletion. However, it is difficult to analyze the runtime of hash tables so we will not use them in this course.